جامعة الأميرة
نورة بنت عبد الرحمن
كلية علوم الحاسب والمعلومات

Princess Nourah bint
Abdulrahman University
College of Computer and
Information Sciences

SmartHire: Automated Resume Parsing System

Phase 5: Final Report

Group no. 5

Section: 62U

No.	Student Name	Student Number
1	Rahaf Saad Eddin	445008916
2	Mace Abdullah Almarzoqi	445008865
3	Renad Bandar Hazazi	445008912
4	Ghala Mohammed Hazazi	445008868
5	Deem Mohammed Alrashoud	445008860

Table Of Contents

Abstract 3

1. Introduction 3

1.1 Project Development Across Phases 3

2. Background and Related Work 4

3. Dataset Description and Analysis 6

3.1 Final Dataset Characteristics 6

4. Methodology 7

4.1 Preprocessing 7

4.2 Model Architecture 7

4.2.1 Baseline Model 7

4.2.2 Proposed Model: Section-Aware Parsing 8

4.2.3 Deployment Strategy 8

4.3 System Pipeline Overview 9

4.4 Training Strategy 9

4.5 Evaluation Strategy 9

5. System Design and Final Implementation 10

5.1 Interface Pipeline 10

5.2 From Raw Text to Candidate Profile 10

5.3 Interface Design 11

5.3.1 Resume Interface 12

5.3.2 Candidate Profile and Resume Intelligence 12

5.3.3 Interview Interface 13

5.3.4 Settings and User Control 13

5.4 Post-processing and Structuring Layer 13

6. Experiments 14

7. Results 14

8. Discussion 14

9. Limitations and Future Work 15

10. Conclusion 15

11. References 16

12. Appendix 17

Abstract

This project presents an automated resume parsing system based on transformer-based Natural Language Processing techniques. The main objective is to extract structured information such as personal details, education, skills, and work experience from unstructured resume text.

In this work, we implement a BERT-based Named Entity Recognition (NER) model as a baseline approach, where the entire resume is processed as a single sequence. In addition, we propose a section-aware parsing approach that incorporates document structure by dividing resumes into logical sections such as education, experience, and skills before applying the model.

To evaluate the effectiveness of both approaches, a comparative analysis is conducted between the baseline model and the section-aware model using standard evaluation metrics, including precision, recall, F1-score, and accuracy. This comparison allows us to examine the impact of structural information on entity extraction performance.

The results demonstrate that the section-aware approach achieves better overall performance by improving contextual understanding and maintaining more consistent extraction across different resume formats.

The final system demonstrates that combining contextual models with structured processing and interface design significantly enhances the usability and effectiveness of resume parsing systems.

1. Introduction

Resume parsing is a practical NLP task that transforms unstructured resumes into machine-readable information. It matters because recruiters work with many applications and need faster ways to identify education, experience, skills, and contact details. In practice, this task is difficult because resumes are only semi-structured. They contain similar information, but the order, layout, style, and wording change from one document to another.

The goal of this project was to build a complete system rather than only train a model. The final application accepts real resumes, extracts their text, parses them with BERT, builds a structured candidate profile, and then reuses that profile inside a chat-based interview assistant. The system was therefore designed to connect model performance with practical usefulness.

1.1 Project Development Across Phases

Phase 1 defined the problem, user needs, and project scope. The focus was to design a system that reduces manual resume review and turns unstructured resumes into useful structured data.

Phase 2 reviewed related academic work in hierarchical labeling, structure-aware parsing, and BERT-based recruitment systems. This phase guided the choice of a contextual model and supported the idea that document structure matters.

Phase 4 formalized the experimental plan. It described the dataset, preprocessing pipeline, baseline model, proposed section-aware model, training setup, and evaluation metrics.

2. Background and Related Work

Recent research in resume parsing has moved beyond rule-based methods toward approaches that combine contextual understanding and structural awareness. Due to the variability in resume formats, modern solutions focus on models that can capture both semantic meaning and document structure.

In this section, we review three representative studies: hierarchical sequence labeling, a multi-segment LLM ensemble framework, and a hybrid KNN-BERT approach.

2.1 Hierarchical Sequence Labeling for Resume Parsing

This study reformulates resume parsing as a hierarchical sequence labeling problem, aiming to improve both structural understanding and entity extraction. Traditional systems often separate the process into two stages: first segmenting the resume into sections, and then extracting entities from each section. However, this approach can increase system complexity and lead to error propagation.

To address this limitation, the authors propose a multi-task learning architecture based on a BiRNN + CRF model. The model performs both line-level segmentation and token-level entity recognition simultaneously. This unified approach allows better integration between document structure and semantic information.

The experimental results show that the multi-task model outperforms single-task approaches, achieving F1 scores between 90% and 92%. This demonstrates that combining structural segmentation with entity extraction significantly improves performance.

This work is highly relevant to our project, as it highlights the importance of incorporating document structure into the parsing process, which directly motivates our section-aware approach.

2.2 Multi-Segment LLM Ensemble Framework (MSLEF)

This study introduces a segment-aware ensemble framework designed to improve resume parsing accuracy across diverse formats. The authors argue that a single model may not perform equally well across all resume sections, and therefore propose using multiple specialized models.

The MSLEF framework integrates several fine-tuned large language models, each focusing on specific parts of the resume. The system generates outputs from each model, normalizes the extracted information, and combines the results using weighted voting and aggregation techniques.

The results show that the ensemble approach achieves an F1 score of 92.6% and improves overall robustness compared to single-model systems. This demonstrates the effectiveness of combining multiple models to handle variations in resume structure and content.

This work is relevant to our project as it emphasizes the importance of segment-aware processing and structured aggregation, which aligns with our goal of improving extraction quality using a section-aware parsing strategy.

2.3 Hybrid KNN-BERT Resume Parsing System

This study proposes a hybrid approach that combines classical machine learning techniques with transformer-based models to improve both parsing accuracy and efficiency. The system integrates K-Nearest Neighbors (KNN) with BERT embeddings to capture both similarity-based and contextual information.

The methodology includes preprocessing steps such as tokenization and normalization, followed by feature extraction using TF-IDF and BERT embeddings. KNN is then used to compute similarity scores between resumes and job descriptions, while a weighted combination of predictions is applied to improve ranking performance.

The results show that this hybrid approach achieves an accuracy of 96.91% and an F1 score close to 97%, outperforming traditional methods while maintaining computational efficiency.

This study is relevant to our project as it demonstrates the benefits of combining contextual understanding with efficient computation, which is important for building practical and scalable resume parsing systems.

2.4 Summary and Motivation

Overall, the reviewed studies demonstrate a clear shift from traditional keyword-based methods to more advanced approaches that integrate contextual embeddings, structural modeling, and ensemble techniques. These methods aim to improve both accuracy and robustness when handling diverse resume formats.

These insights directly influence the design of our system, which combines a BERT-based NER model with a section-aware parsing approach to better capture both the semantic and structural aspects of resumes.

Study	Core Idea	Why It Matters Here
Hierarchical sequence labeling [1]	Joint structure and entity modeling	Encouraged a structure-aware parsing strategy.
Multi-segment LLM ensemble [2]	Segment-specific extraction for recruitment	Showed that local document structure improves understanding.
Hybrid KNN-BERT parser [3]	Contextual embeddings plus practical logic	Supported combining BERT with post-processing rather than relying on a model alone.

These insights directly influence the design of our system, which combines a BERT-based NER model with a section-aware parsing approach to better capture both the semantic and structural aspects of resumes.

3. Dataset Description and Analysis

The project used two data stages. The early design work used a smaller pilot dataset of around 220 resumes. The final implementation used a larger merged NER dataset with 5,960 resumes and a wider range of entity types. This second stage gave the model more exposure to noise, layout variety, and real-world resume language.

Dataset Stage	Resumes	Annotated Entities	Entity Types	Average Length	Purpose
Pilot experimental dataset	~220	~3,556	11	~3,697 chars	Initial design and experiment planning
Final merged dataset	5960	579572	14	4447.51	Final training and deployment

https://www.kaggle.com/datasets/yashpwrr/resume-ner-training-dataset

3.1 Final Dataset Characteristics

The final dataset contains 5960 resumes and 579572 annotated spans. The average resume length is 4447.51 characters and the maximum length is 99973 characters. The dataset is highly imbalanced because skill labels appear far more often than company, certification, or language labels. The most frequent labels are SKILL (549462), OTHER (10267), DESIGNATION (4301), LOCATION (4073), EXPERIENCE (3544), PERSON (3122), EDUCATION (2124), EXPERTISE (1045). This imbalance influenced the training strategy, the use of class weights, and the interpretation of the final metrics.

4. Methodology

4.1 Preprocessing

Step	Implementation	Purpose
Annotation cleanup	Trim spans and remove overlap	Avoid contradictory labels before training.
BIO conversion	Convert spans into token-level B/I/O labels	Required for BERT token classification.
Tokenization	Use the tokenizer linked to the chosen BERT checkpoint	Keep train and test formatting aligned.
Long-sequence chunking	Split long resumes with overlap	Handle documents longer than the BERT token limit.
Section splitting	Detect Education, Experience, Skills, Projects, and Language blocks	Support the proposed section-aware parser.

Preprocessing was not only a training step. It also shaped deployment quality. The final project learned that PDF resumes become useful only after text normalization, spacing repair, and section repair. This is why preprocessing appears both before training and before live inference.

4.2 Model Architecture

To systematically evaluate our approach, we adopt a baseline vs. proposed model design.

Both models are built on the same pretrained bert-base-cased backbone and are evaluated on the same prediction task.

4.2.1 Baseline Model

The baseline model is a standard BERT-based token classification model. We use the pretrained bert-base-cased model and fine-tune it on our dataset for token-level BIO labeling of resume entities.

In this setup:

The entire resume is treated as a single continuous sequence

The model predicts entity labels for each token

No structural information is explicitly considered

This approach serves as a reference model and represents a widely used method in NER tasks. It allows us to establish a performance baseline against which improvements can be measured.

4.2.2 Proposed Model: Section-Aware Parsing

The proposed model introduces a structural enhancement to the baseline by incorporating document segmentation, while retaining the same bert-base-cased backbone.

Instead of processing the resume as a flat sequence, the system first divides the document into logical sections such as:

- Education
- Experience
- Skills

This segmentation can be achieved using simple heuristics or rule-based detection of section headers.

Once segmented, each section is processed independently using the same BERT model. This allows the model to focus on localized context within each section, which can improve entity recognition.

For example:

The word “Python” in the Skills section likely refers to a skill

The same word in Experience may have a different contextual meaning

By isolating sections, the model can better interpret entity roles and reduce ambiguity.

This approach aligns with hierarchical and structure-aware methods discussed in prior research, where document segmentation improves information extraction by preserving contextual boundaries.

4.2.3 Deployment Strategy

For deployment, we adopt a comparative approach by keeping both the baseline and the proposed parsers available.

Each uploaded resume is processed by both models, and their outputs can be compared to analyzing performance differences and validate improvements.

4.3 System Pipeline Overview

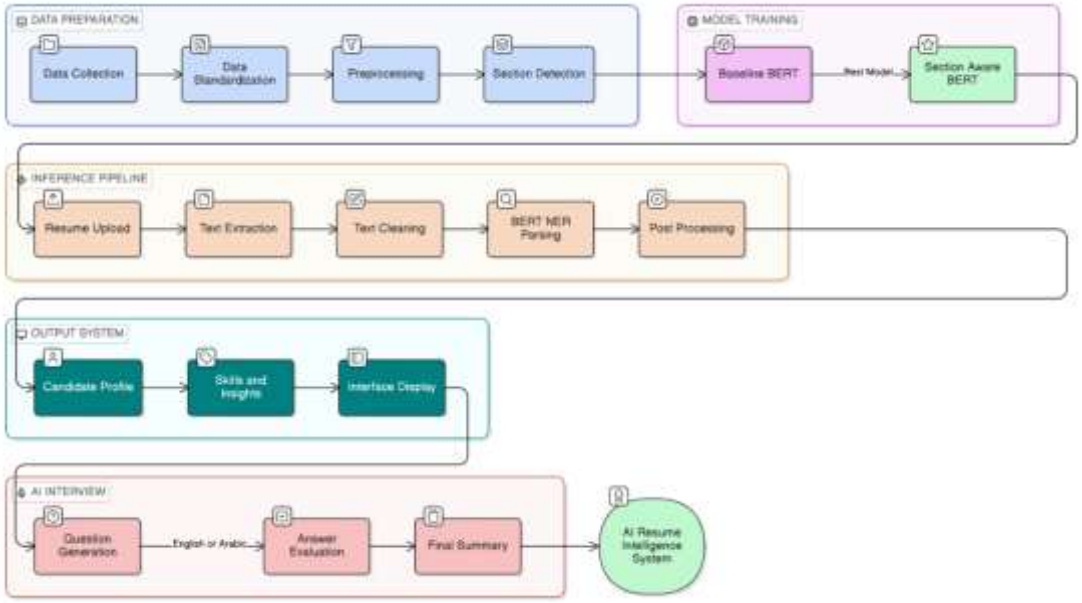


Figure 1: End-to-End System Pipeline of SmartHire: Automated Resume Parsing System.

4.4 Training Strategy

Training Element	Final Setting
Backbone model	bert-base-cased
Epochs	5
Max length	128
Batch size	16
Loss	Cross-entropy with class weights
Early stopping	Enabled using validation F1-score
Main comparison metric	Token-level micro F1 on non-O labels

4.5 Evaluation Strategy

The evaluation stage used both token-level and entity-level metrics. At the token level, precision, recall, F1-score, and accuracy were computed over BIO labels. This can be considered a partial evaluation because it measures correctness at the token level even when a complete entity span is not fully correct. .

At the entity level, exact-match precision, recall, and F1-score were used. In this setting, an entity is counted as correct only if its full boundaries and label are predicted exactly. Section-level accuracy was also included to assess whether predictions remained consistent within the structural sections of the resume. .

F1-score was treated as the main comparison metric because the label "O" is highly frequent and can make accuracy misleadingly high. Therefore, token-level micro F1 on non-"O" labels provided the most balanced measure for model comparison.

5. System Design and Final Implementation

5.1 Interface pipeline

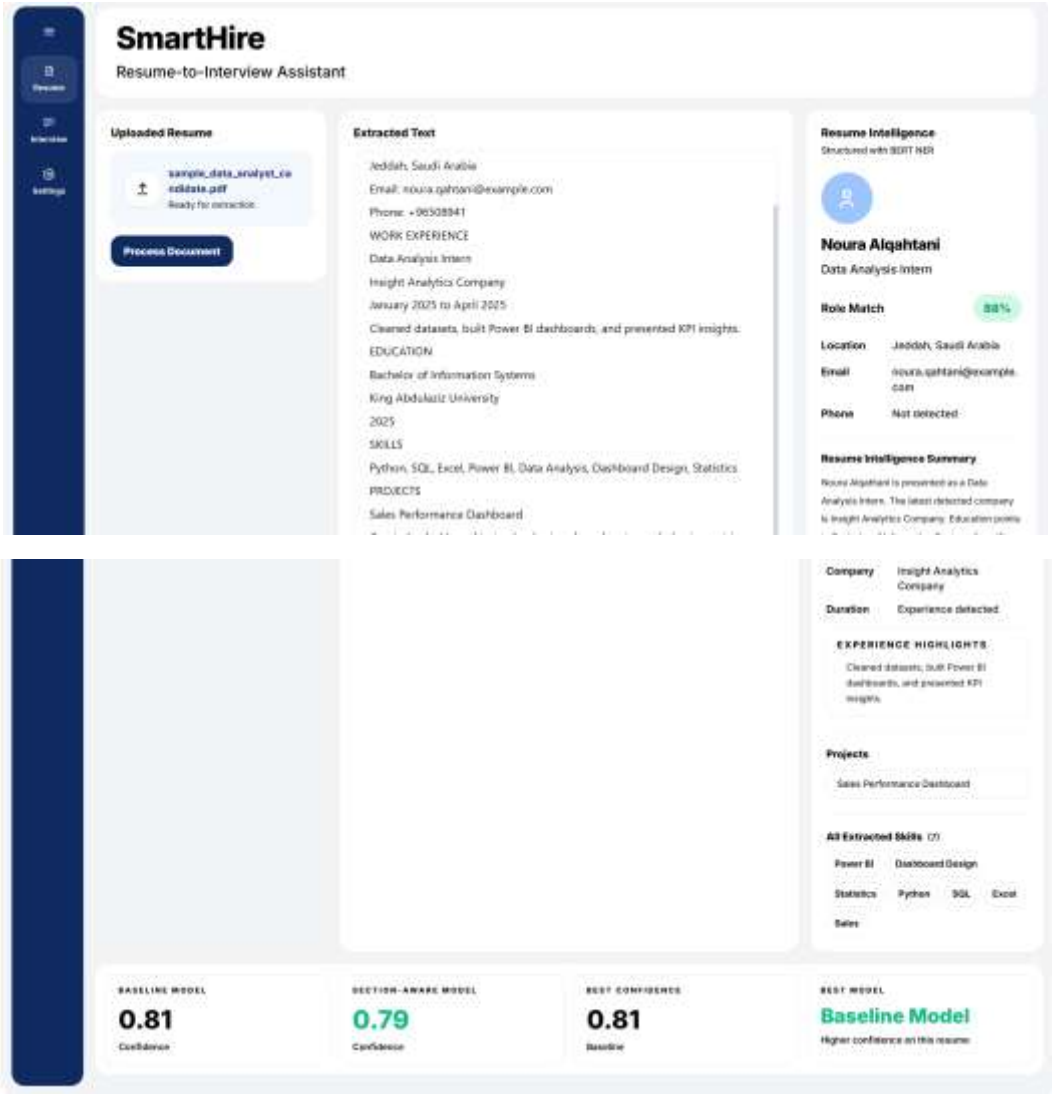
The final system starts when the user uploads a PDF, DOCX, or TXT resume. The server extracts raw text from the document, normalizes the text, converts it into a training-aligned representation, and then passes it to the selected BERT parser. The predicted entities are then transformed into structured fields through a post-processing layer. The resulting candidate profile is displayed in the interface and reused by the interview assistant.

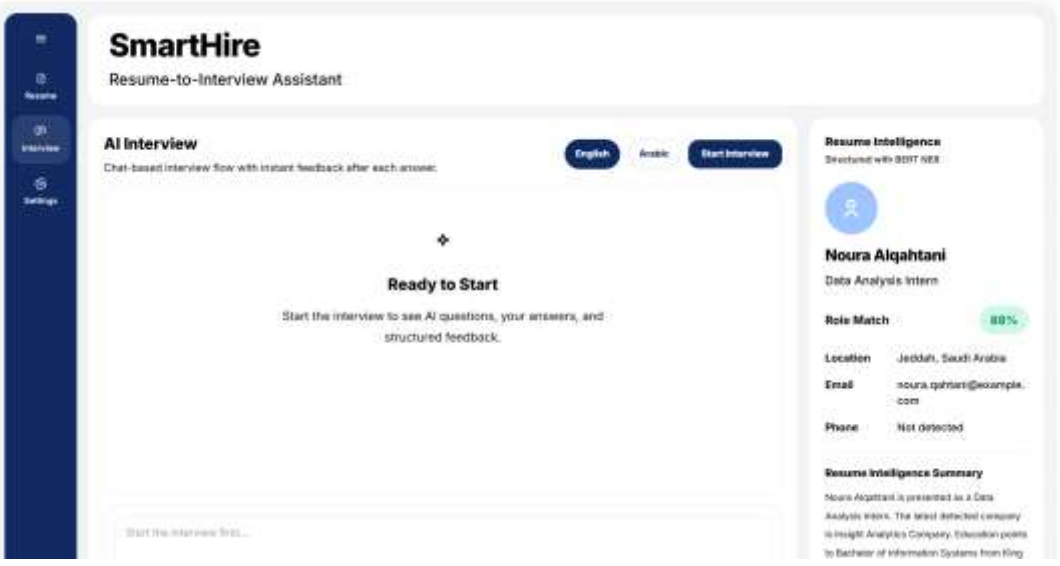
5.2 From Raw Text to Candidate Profile

A major contribution of the final implementation is the intelligence layer between raw model output and the user interface. Real resumes often contain layout noise, duplicated headers, multi-column ordering problems, repeated contact strings, and mixed formatting from visual templates. The project therefore includes cleaning and restructuring steps after BERT prediction. This layer merges duplicate values, normalizes skills, repairs obvious contact fields, improves role titles, separates education from work content, and finally presents the result as a structured candidate profile.

Transformation Stage	What the System Does
Raw text	Extracts resume text from PDF, DOCX, or TXT input.
Normalized text	Repairs spacing, line breaks, and repeated headers.
BERT output	Predicts token-level entities such as person, skill, education, company, email, and location.
Post-processing	Merges entities into recruiter-facing fields and removes noise.
Candidate profile	Displays the final structured information in the interface.

5.3 Interface Design





Interface Area	Function	Displayed Information
Resume page	Document parsing view	Uploaded file, extracted raw text, Resume Intelligence output
Interview page	Chat-based evaluation	Questions, answers, feedback after each answer, final summary
Settings page	System control	Language, theme, and model preferences

The interface was designed to make the system feel like a real product instead of a notebook output. Rather than exposing only raw model predictions, the final application presents a guided workflow with a sidebar, a main processing panel, and a structured information panel. This design helps the user understand not only what the model predicted, but also how those predictions are turned into useful decisions.

5.3.1 Resume Interface

The Resume page is the entry point of the system. It allows the user to upload a PDF, DOCX, or TXT file, inspect the extracted raw text, and then review the final Resume Intelligence output. This page is important because it shows both stages clearly: what the document extractor read, and what the parsing system understood from that text.

5.3.2 Candidate Profile and Resume Intelligence

The right-side information panel is the most important part of the interface. It presents the parsed resume as a structured candidate profile. The final design includes name, title, role match, contact information, education, experience, languages, and extracted skills. This panel was intentionally designed to behave like a recruiter-facing summary rather than a raw technical output. In other words, the interface translates NER results into a format that can support screening and decision making.

Resume Intelligence Element	Why It Was Included
Name and title	Provide the candidate's identity and current role in the first view.
Role match	Give a quick summary score that supports fast screening.
Contact block	Show whether email, phone, and location were successfully extracted.
Education block	Present university, major, qualification, and graduation year in one place.
Experience block	Summarize current role, company, and extracted highlights.
Skills and languages	Show the practical information that supports both matching and interview generation.

5.3.3 Interview Interface

The Interview page uses a chat-based layout to simulate a realistic AI interview assistant. After the resume is parsed, the extracted candidate profile is passed to the interview module as structured context. Based on this information, the system generates interview questions that are relevant to the candidate's background, skills, and experience. This makes the interview stage more meaningful than a static parser because it demonstrates how extracted resume data can support downstream NLP tasks such as question generation and answer evaluation.

The interview component was implemented using an API-based large language model to generate questions, evaluate answers, and produce follow-up feedback. The system supports both English and Arabic interview modes. After each answer, the model returns structured feedback including a score, strengths, areas for improvement, and a suggested stronger answer. A final summary is also generated at the end of the interview to provide an overall assessment of the candidate's performance. To improve system reliability, a fallback interview logic was also included when the external API is unavailable.

5.3.4 Settings and User Control

The Settings page provides user control over language, theme, and system preferences. This page is not only visual convenience. It supports usability, especially when the system is presented as a complete application. The language selection is especially important because the project supports both English and Arabic interview flows.

Overall, the final interface acts as the bridge between model output and user trust. A strong model is useful, but a clean and interpretable interface is what allows the model to become a professional application.

5.4 Post-processing and Structuring Layer

The post-processing stage was necessary because a correct token label does not automatically produce a clean recruiter-facing field. For example, education text must be split into university, major, qualification, and graduation year. Skills must be merged, cleaned, deduplicated, and displayed without noisy fragments. Contact values need formatting repair. This layer made the final system useful in practice and turned raw NER output into information that a recruiter can read quickly.

· Contact cleanup: repair email and phone formatting.

· Education parsing: separate university, major, qualification, and graduation year.

· Skill normalization: remove obvious noise and combine duplicate skill mentions.

· Profile assembly: build Resume Intelligence from raw entities and fallback evidence in the text.

6. Experiments

The experiments compared the baseline and the section-aware model under the same general conditions. The goal was to test whether document structure improves extraction quality in a measurable way and whether that improvement remains meaningful after deployment on real resumes.

Precision: Measures of how many of the predicted entities are actually correct.

Recall: Measures of how many of the real entities were successfully identified.

F1-score: Balances precision and recall; used as the main comparison metric.

Accuracy: Measures overall token correctness but is less reliable on its own for NER.

Entity-level F1: Measures performance based on exact matches of complete entity spans.

Section-level accuracy: Measures whether section-aware parsing preserves the document structure correctly.

7. Results

Metric	Baseline	Section-Aware
Precision	0.6792	0.6860
Recall	0.9102	0.9068
F1-score	0.7779	0.7811
Accuracy	0.9236	0.9273
Entity F1	0.7692	0.7662
Section-level Accuracy	0.9236	0.9381

The final section-aware parser achieved the strongest overall result, with an F1-score of 0.7811 and an accuracy of 0.9273. The baseline model remained highly competitive and performed slightly better on exact entity span F1. This means the proposed method improved overall balance and structural consistency, while the baseline remained slightly safer on some exact boundaries.

8. Discussion

The final results show that structure does improve extraction, but the real success of the project came from combining several layers: extraction, normalization, BERT parsing, post-processing, and interface design. The model alone is important, but a practical system needs a full pipeline around it.

Another important lesson is that raw NER output is not the same as recruiter-ready information. A model may correctly detect education text, but a useful system must still separate that text into university, qualification, major, and graduation year. It must also present the result in a way that is clear inside an application interface. The Resume Intelligence layer solved this gap.

9. Limitations and Future Work

· Very noisy or highly artistic PDF resumes still reduce extraction quality.

· The current system uses strong extraction heuristics, but OCR fallback would help image-heavy resumes.

· Skill extraction can be improved further through stronger normalization and skill grouping.

· Future work should include multilingual support and stronger confidence calibration.

· Role match can be improved by adding explicit job-description matching instead of resume-only heuristics.

10. Conclusion

This project produced a complete automated resume parsing system rather than only a trained model. The final solution combines a BERT-based NER parser, a section-aware improvement, a resume intelligence layer, a recruiter-oriented interface, and an interview assistant. The section-aware model achieved the strongest overall result, and the full pipeline showed that structured resume intelligence becomes much more useful when raw extraction is converted into clear, organized candidate information. In short, the project demonstrates that real resume parsing quality depends on both a strong model and a carefully designed pipeline around that model.

11. References

[1] F. Retyk, H. Fabregat, J. Aizpuru, M. Taglio, and R. Zbib, "Resume Parsing as Hierarchical Sequence Labeling: An Empirical Study," 2023.

[2] O. Walid, M. T. Younes, K. Shaban, M. Hassan, and A. Hamdi, "MSLEF: Multi-Segment LLM Ensemble Finetuning in Recruitment," 2025.

[3] O. E. Olorunshola, I. O. Ampitan, F. Adamu-Fika, and A. K. Ademuwagun, "An Enhanced K-NN Algorithm Leveraging BERT Techniques for Resume Parsing System," 2025.

[4] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," 2019.

[5] Hugging Face, "Transformers Documentation," n.d. [Online]. Available: https://huggingface.co/docs/transformers

[6] FastAPI, "FastAPI Documentation," n.d. [Online]. Available: https://fastapi.tiangolo.com/

12. Appendix

Appendix A: Data Preprocessing

File: preprocessing.py

```
from dataclasses import dataclass
import re

SECTION_HEADERS = {
    "education": ["EDUCATION", "ACADEMIC BACKGROUND", "QUALIFICATION"],
    "experience": ["WORK EXPERIENCE", "EXPERIENCE", "EMPLOYMENT HISTORY"],
    "skills": ["SKILLS", "TECHNICAL SKILLS", "ADDITIONAL SKILLS"],
    "projects": ["PROJECTS", "PROJECT EXPERIENCE"],
    "language": ["LANGUAGES", "LANGUAGE"],
    "certifications": ["CERTIFICATIONS", "CERTIFICATES", "CERTIFICATES &
GRANTS"],
}



@dataclass
class TextSegment:
    """A piece of resume text that will be sent to BERT."""
    text: str
    start_offset: int
    section_name: str



def clean_text_for_display(text: str) -> str:
    """Clean text lightly for printing output."""
")-" ,"¢Ï¢â,¬Í    text = text.replace("
    text = re.sub(r"\n{3,}", "\n\n", text)
    return text.strip()



def build_label_list(records: list) -> list[str]:
    """Create BIO labels from all entity types in the dataset."""
    entity_labels = sorted({entity.label for record in records for entity in record.entities})
    labels = ["O"]
```

```
    for label in entity_labels:
        labels.append(f"B-{label}")
        labels.append(f"I-{label}")
    return labels
```

Appendix B: BIO Label Conversion

File: preprocessing.py

```
def convert_entities_to_bio(
    text: str,
    entities: list,
    offsets: list[tuple[int, int]],
    label_to_id: dict[str, int],
    segment_start: int = 0,
) -> list[int]:
    """Convert character-level spans to token-level BIO labels."""
    labels = [
        -100 if token_start == token_end else label_to_id["O"]
        for token_start, token_end in offsets
    ]
    sorted_entities = sorted(entities, key=lambda item: (item.start, item.end))

    for entity in sorted_entities:
        entity_start = entity.start - segment_start
        entity_end = entity.end - segment_start + 1
        first_entity_token = True

        for token_index, (token_start, token_end) in enumerate(offsets):
            if token_start == token_end:
                continue

            overlaps_entity = token_start < entity_end and token_end > entity_start
            if not overlaps_entity:
                continue

            prefix = "B" if first_entity_token else "I"
            labels[token_index] = label_to_id[f"{prefix}-{entity.label}"]
            first_entity_token = False
```

```
    return labels
```

Appendix C: Section-Aware Parsing

File: preprocessing.py

```
def split_resume_into_sections(text: str) -> list[TextSegment]:
    """Split a resume using simple section-header rules."""
    matches = []
    for section_name, headers in SECTION_HEADERS.items():
        for header in headers:
            pattern = rf"(?im)^\s*{re.escape(header)}\s*$"
            for match in re.finditer(pattern, text):
                matches.append((match.start(), section_name))

            spaced_pattern = _build_spaced_header_pattern(header)
            for match in re.finditer(spaced_pattern, text):
                matches.append((match.start(), section_name))

    if not matches:
        for section_name, headers in SECTION_HEADERS.items():
            for header in headers:
                pattern = rf"(?i)(?<![A-Za-z]){re.escape(header)}(?![A-Za-z])"
                for match in re.finditer(pattern, text):
                    matches.append((match.start(), section_name))

    if not matches:
        return [TextSegment(text=text, start_offset=0, section_name="unknown")]

    matches = sorted(set(matches))
    segments: list[TextSegment] = []

    if matches[0][0] > 0:
        segments.append(
            TextSegment(text=text[: matches[0][0]], start_offset=0, section_name="header")
        )

    for index, (start, section_name) in enumerate(matches):
```

```
        end = matches[index + 1][0] if index + 1 < len(matches) else len(text)
        segment_text = text[start:end]
        if segment_text.strip():
            segments.append(
                TextSegment(text=segment_text, start_offset=start,
section_name=section_name)
            )

    return segments
```

Appendix D: BERT Model Definition

File: models.py

```
import torch
from transformers import AutoModelForTokenClassification, AutoTokenizer

MODEL_NAME = "bert-base-cased"


def load_tokenizer(model_name: str = MODEL_NAME):
    """Load the BERT tokenizer used for token-level NER."""
    return AutoTokenizer.from_pretrained(model_name)


def create_bert_ner_model(
    label_list: list[str],
    model_name: str = MODEL_NAME,
):
    """Create a BERT token-classification model."""
    id_to_label = {index: label for index, label in enumerate(label_list)}
    label_to_id = {label: index for index, label in id_to_label.items()}

    model = AutoModelForTokenClassification.from_pretrained(
        model_name,
        num_labels=len(label_list),
        id2label=id_to_label,
        label2id=label_to_id,
    )
    return model
```

```
def get_device() -> torch.device:
    """Use GPU if available, otherwise CPU."""
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Appendix E: Training Loop

File: training.py

```
def train_model(
    model,
    train_features: list[dict],
    validation_features: list[dict],
    device,
    id_to_label: dict[int, str],
    output_dir,
    epochs: int = 3,
    batch_size: int = 16,
    learning_rate: float = 2e-5,
    patience: int = 2,
    use_class_weights: bool = True,
    class_weight_max: float = 5.0,
) -> dict[str, float]:
    """Fine-tune BERT and keep the best model based on validation F1."""
    output_path = Path(output_dir)
    output_path.mkdir(parents=True, exist_ok=True)

    train_loader = create_data_loader(train_features, batch_size=batch_size,
shuffle=True)
    validation_loader = create_data_loader(
        validation_features, batch_size=batch_size, shuffle=False
    )

    model.to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

    if use_class_weights:
        class_weights = compute_class_weights(
            train_features,
```

```
        id_to_label,
        max_weight=class_weight_max,
    ).to(device)
    loss_function = torch.nn.CrossEntropyLoss(
        weight=class_weights,
        ignore_index=-100,
    )
else:
    loss_function = torch.nn.CrossEntropyLoss(ignore_index=-100)

best_f1 = -1.0
best_metrics = {}
epochs_without_improvement = 0

for epoch in range(1, epochs + 1):
    model.train()
    total_loss = 0.0

    for batch in train_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        optimizer.zero_grad()
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
        )
        logits = outputs.logits
        loss = loss_function(
            logits.view(-1, logits.shape[-1]),
            labels.view(-1),
        )
        loss.backward()
        optimizer.step()

        total_loss += float(loss.item())
```

```
        average_loss = total_loss / max(len(train_loader), 1)
        metrics = evaluate_model(model, validation_loader, device, id_to_label)

        print(
            f"Epoch {epoch}: loss={average_loss:.4f}, "
            f"val_precision={metrics['precision']:.4f}, "
            f"val_recall={metrics['recall']:.4f}, "
            f"val_f1={metrics['f1']:.4f}"
        )

        if metrics["f1"] > best_f1:
            best_f1 = metrics["f1"]
            best_metrics = metrics
            epochs_without_improvement = 0
            model.save_pretrained(output_path)
        else:
            epochs_without_improvement += 1

        if epochs_without_improvement >= patience:
            print("Early stopping: validation F1 did not improve.")
            break

    return best_metrics
```

Appendix F: Evaluation Metrics

File: evaluation.py

```
from sklearn.metrics import precision_recall_fscore_support

def compute_token_metrics(
    true_label_ids: list[int],
    predicted_label_ids: list[int],
    id_to_label: dict[int, str],
) -> dict[str, float]:
    """Compute token-level precision, recall, and F1."""
    filtered_true = []
    filtered_predicted = []
```

```
    for true_id, predicted_id in zip(true_label_ids, predicted_label_ids):
        if true_id == -100:
            continue
        filtered_true.append(id_to_label[int(true_id)])
        filtered_predicted.append(id_to_label[int(predicted_id)])

    labels_without_o = [label for label in id_to_label.values() if label != "O"]

    precision, recall, f1, _ = precision_recall_fscore_support(
        filtered_true,
        filtered_predicted,
        labels=labels_without_o,
        average="micro",
        zero_division=0,
    )

    correct_tokens = sum(
        true_label == predicted_label
        for true_label, predicted_label in zip(filtered_true, filtered_predicted)
    )
    token_accuracy = correct_tokens / len(filtered_true) if filtered_true else 0.0

    return {
        "precision": float(precision),
        "recall": float(recall),
        "f1": float(f1),
        "accuracy": float(token_accuracy),
    }
def evaluate_model(
    model,
    data_loader,
    device,
    id_to_label: dict[int, str],
) -> dict[str, float]:
    """Run the model on a dataset and return evaluation metrics."""
    model.eval()
    all_true = []
    all_predicted = []
```

```
    with torch.no_grad():
        for batch in data_loader:
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels = batch["labels"].to(device)

            outputs = model(input_ids=input_ids, attention_mask=attention_mask)
            predictions = torch.argmax(outputs.logits, dim=-1)

            for true_sequence, predicted_sequence in zip(labels.cpu(), predictions.cpu()):
                true_list = true_sequence.tolist()
                predicted_list = predicted_sequence.tolist()

                for true_id, predicted_id in zip(true_list, predicted_list):
                    if true_id != -100:
                        all_true.append(true_id)
                        all_predicted.append(predicted_id)

    return compute_token_metrics(all_true, all_predicted, id_to_label)
```

Appendix G: PDF / DOCX Extraction Pipeline

File: document_loader.py

```
from pathlib import Path

def extract_text_from_document(file_path: str | Path) -> str:
    """Extract raw text from PDF, DOCX, TXT, or JSON files."""
    path = Path(file_path)
    suffix = path.suffix.lower()

    if suffix == ".pdf":
        return _normalize_extracted_text(_extract_pdf_text(path))
    if suffix == ".docx":
        return _normalize_extracted_text(_extract_docx_text(path))
    if suffix in {".txt", ".json"}:
        return _normalize_extracted_text(path.read_text(encoding="utf-8",
errors="ignore").strip())
```

```
    raise ValueError("Supported file types: PDF, DOCX, TXT, JSON")
def _extract_pdf_text(path: Path) -> str:
    """Extract text from a PDF file using multiple extractors."""
    candidates = []

    pypdf_text = _extract_pdf_text_with_pypdf(path)
    if pypdf_text:
        candidates.append(("pypdf", pypdf_text))

    pdfplumber_text = _extract_pdf_text_with_pdfplumber(path)
    if pdfplumber_text:
        candidates.append(("pdfplumber", pdfplumber_text))

    pymupdf_text = _extract_pdf_text_with_pymupdf(path)
    if pymupdf_text:
        candidates.append(("pymupdf", pymupdf_text))

    if not candidates:
        return ""

    best_name, best_text = max(candidates, key=lambda item:
_score_extracted_text(item[1]))
    return best_text
```

Appendix H: Post-processing and Candidate Profile Construction

File: candidate_profile.py

```
def build_profile_from_entities(entities: dict[str, list[str]]) -> dict[str, list[str]]:
    """Create a clean candidate profile from extracted resume entities."""
    profile: dict[str, list[str]] = {}

    for field_name in PROFILE_FIELDS:
        values = entities.get(field_name, [])
        output_field_name = LABEL_ALIASES.get(field_name, field_name)

        if field_name in {"Skills", "SKILL", "SKILLS", "EXPERTISE"}:
            cleaned_values = _clean_skill_values(values)
```

```
        elif output_field_name == "Languages":
            cleaned_values = _clean_language_values(values)
        elif output_field_name == "Name":
            cleaned_values = _clean_name_values(values)
        elif output_field_name == "Email Address":
            cleaned_values = _clean_email_values(values)
        elif output_field_name == "Designation":
            cleaned_values = _clean_designation_values(values)
        elif output_field_name == "Degree":
            cleaned_values = _clean_degree_values(values)
        else:
            cleaned_values = []
            for value in values:
                clean_value = _clean_general_value(str(value))
                if clean_value and clean_value not in cleaned_values:
                    cleaned_values.append(clean_value)

        if cleaned_values:
            existing_values = profile.setdefault(output_field_name, [])
            for clean_value in cleaned_values:
                if clean_value not in existing_values:
                    existing_values.append(clean_value)
            profile[output_field_name] = existing_values[:8]

    return profile
def _clean_skill_values(values: list[str]) -> list[str]:
    """Split long skills sections into short readable skill names."""
    cleaned_skills = []

    for value in values:
"),"  ,"¢Ï¢â‚¬¬ï        text = str(value).replace("â€¢", ",").replace("
        text = re.sub(r"\bNon\s*-\s*Technical Skills\b.*", "", text, flags=re.IGNORECASE)
        text = re.sub(r"\([^)]*\)", "", text)
        parts = re.split(r"[,;\n]+", text)

        for part in parts:
            skill = " ".join(part.split())
            if ":" in skill:
```

```
            skill = skill.split(":", 1)[1].strip()

        skill = skill.strip(" .:-")
        if not skill:
            continue
        if len(skill) <= 1:
            continue
        if len(skill) > 40:
            continue
        if skill not in cleaned_skills:
            cleaned_skills.append(skill)

    return cleaned_skills[:12]
```

Appendix I: System Integration (Upload + Extraction + BERT + UI Output)

File: app_server.py

```
@app.post("/api/process-resume")
def process_resume(
    file: UploadFile = File(...),
    parser_model: str = BEST_PARSER_KEY,
) -> dict[str, Any]:
    saved_path: Path | None = None
    try:
        saved_path = save_upload(file)
        raw_text = extract_text_from_document(saved_path)
        if not raw_text.strip():
            raise HTTPException(status_code=400, detail="No text could be extracted from this document.")

        bundle = get_parser_bundle(parser_model)
        baseline_bundle = get_parser_bundle("baseline")
        section_bundle = get_parser_bundle("section_aware")

        baseline_entities, baseline_confidence = predict_entities_with_quality(
            resume_text=raw_text,
            model=baseline_bundle["model"],
            tokenizer=baseline_bundle["tokenizer"],
```

```
            device=DEVICE,
            max_length=MAX_LENGTH,
            section_aware=baseline_bundle["section_aware"],
        )
        section_entities, section_confidence = predict_entities_with_quality(
            resume_text=raw_text,
            model=section_bundle["model"],
            tokenizer=section_bundle["tokenizer"],
            device=DEVICE,
            max_length=MAX_LENGTH,
            section_aware=section_bundle["section_aware"],
        )

        if bundle["key"] == "baseline":
            entities = baseline_entities
            model_confidence = baseline_confidence
        else:
            entities = section_entities
            model_confidence = section_confidence

        profile = merge_profile_with_fallback(build_profile_from_entities(entities), raw_text)
        ui_profile = build_ui_profile(profile, model_confidence=model_confidence)

        return {
            "processed": True,
            "extractedText": raw_text,
            "profile": ui_profile,
            "profileRaw": profile,
            "stats": build_stats(profile),
            "metrics": build_resume_confidence_cards(baseline_confidence,
section_confidence),
            "parserModel": bundle["key"],
            "parserLabel": "Section-Aware BERT" if bundle["key"] == "section_aware" else
"Baseline BERT",
            "fileName": file.filename,
        }
    except HTTPException:
        raise
```

```
    except Exception as error:
        raise HTTPException(status_code=500, detail=str(error)) from error
    finally:
        if saved_path and saved_path.exists():
            try:
                saved_path.unlink()
            except OSError:
                pass
```

Appendix J: Interview Integration

File: app_server.py

```
@app.post("/api/start-interview")
def start_interview(request: InterviewStartRequest) -> dict[str, Any]:
    result = generate_questions_safe(
        profile=request.profile,
        language=request.language,
        selected_model=request.selected_model,
        question_count=request.question_count,
    )
    return result


@app.post("/api/evaluate-answer")
def evaluate_interview_answer(request: EvaluateAnswerRequest) -> dict[str, Any]:
    return evaluate_answer_safe(
        question=request.question,
        answer=request.answer,
        profile=request.profile,
        language=request.language,
        selected_model=request.selected_model,
    )


@app.post("/api/final-summary")
def final_summary(request: FinalSummaryRequest) -> dict[str, Any]:
    return create_summary_safe(
        results=request.results,
        language=request.language,
```

```
    selected_model=request.selected_model,
)
```